ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение «ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

КУРСОВОЙ ПРОЕКТ

ПМ.02 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

МДК 02.01 Сопровождение и схемотехническое обслуживание интеллектуальных интегрированных систем

РАЗРАБОТКА И СОПРОВОЖДЕНИЕ ВИРТУАЛЬНОЙ СИСТЕМЫ СБОРА И УЧЁТА ДАННЫХ НА ARDUINO «ВАРИАНТ №20»

Выполнил:

Терехов Д. И.

студент: 54-15 группы

Специальность: 09.02.08 Интеллектуальные интегрированные системы

Руководитель: Борисов Е. Н. Дадаев М. В.

Оценка

Санкт-Петербург, 2026

ПРАВИТЕЛЬСТВО САНКТ-ПЕТЕРБУРГА
КОМИТЕТ ПО НАУКЕ И ВЫСШЕЙ ШКОЛЕ

Санкт-Петербургское государственное бюджетное профессиональное образовательное учреждение
«ПЕТРОВСКИЙ КОЛЛЕДЖ»

(СПб ГБПОУ «Петровский колледж»)

Отделение информационных технологий

УТВЕРЖДАЮ

Заведующий отделением информационных технологий А.Ю. Бычков

_____

«_____» _____ 2026 г.

З А Д А Н И Е

для выполнения курсового проекта

Специальность:	09.02.08 «Интеллектуальные интегрированные системы»
Группа:	54-15
Ф.И.О. студента	Терехов Д. И.

Тема курсового проекта:

Разработка и сопровождение виртуальной системы сбора и учёта данных на Arduino «Вариант №20»

Разработка принципиальной схемы в EasyEDA, схемы на Arduino Uno в симуляторе Wokwi, прошивки, базы данных SQLite, загрузки и контроля данных в Node-RED, ведение проекта в Git. Входные данные варианта (заполняются по таблице вариантов):

Сценарий	Газовая котельная
Датчик 1 (аналоговый)	MQ2 (метан, 0–100 %)
Датчик 2	потенциометр (давление газа, 0–5 бар)
Датчик 3	PIR (движение, 0 или 1)
Исполнительное устройство	реле (газовый клапан)
Светодиод-индикатор (цвет, ток)	белый, 15 мА
Период опроса T	5 с
Порог события	метан > 20 %

Дата выдачи задания для выполнения курсового проекта	«13» октября 2026 г.
Срок защиты курсового проекта	«5» ноября 2026 г.

Содержание

ВВЕДЕНИЕ.....5
1. Постановка задачи.....6
2. Структура и алгоритм работы системы.....7
2.1. Структурная схема.....7
2.2. Алгоритм функционирования.....7
2.3. Протокол обмена.....7
3. Разработка аппаратной части.....8
3.1. Компоненты и подключение.....8
3.2. Схема EasyEDA и модель Wokwi.....8
3.3. Расчёт резистора и шага АЦП.....9
Расчёт токоограничивающего резистора.....9
Расчёт шага АЦП.....10
4. Разработка прошивки.....11
5. Проектирование базы данных.....12
5.1. Модель и структура таблиц.....12
5.2. Контрольные запросы.....12
6. Загрузка и контроль данных в Node-RED.....13
6.1. Состав потока.....13
6.2. Разбор журнала и запуск.....14
6.3. Подготовка и запуск.....15
7. Тестирование и результаты испытаний.....16
7.1. Результаты штатного журнала.....16
7.2. Проверка неисправности и повторной загрузки.....17
8. Расчёт объёма данных.....19
9. Управление версиями Git.....20
10. Предложения по модернизации.....22
Выводы.....23
Список используемых сокращений.....24
Список используемой литературы.....25

Приложение А. Структурная схема и алгоритм.....26
Приложение Б. Схема и спецификация.....27
Приложение В. Схема в Wokwi.....29
Приложение Г. Листинг скетча.....30
Приложение Д. Схема и запросы SQL.....31
Приложение Е. ER-диаграмма.....32
Приложение Ж. Код функции потока Node-RED.....33
Приложение З. Журналы и результаты запросов.....34
Приложение И. История Git и структура репозитория.....36

ВВЕДЕНИЕ

Интеллектуальные интегрированные системы объединяют датчики, микроконтроллер, исполнительные устройства и средства обработки информации. При сопровождении такой системы важно не только получить измерение, но и сохранить его, определить отклонения и восстановить последовательность событий по журналу. Виртуальная модель позволяет изучить эти задачи без монтажа реального оборудования.

Объектом проекта является виртуальная система контроля газовой котельной. Предмет разработки — аппаратная модель на Arduino Uno, программа опроса датчиков, протокол текстового обмена, база SQLite и поток загрузки данных Node-RED. Контролируются учебный показатель содержания метана, давление газа и наличие движения в помещении.

Цель работы — разработать и документировать систему варианта №20, в которой показания трёх датчиков передаются в Serial Monitor, сохраняются в текстовый журнал и загружаются в базу вместе с событиями и признаками неисправности.

Для достижения цели выполнены разработка принципиальной схемы и перечня элементов, сборка модели Wokwi, написание и калибровка прошивки, расчёт резистора, шага АЦП и объёма данных, создание таблиц и запросов SQLite, настройка Node-RED, проверка порога и выхода значения за допустимый диапазон, а также сохранение результатов в Git.

Практическая значимость проекта заключается в воспроизводимой цепочке «измерение — журнал — обработка — хранение». Все файлы проекта опубликованы в открытом репозитории GitHub (ссылка в разделе 9). Полученные файлы позволяют повторить проверку, изменить период или порог и оценить последствия изменения. Система является учебной: она не заменяет сертифицированную систему газовой безопасности и не предназначена для управления реальной котельной.

1. Постановка задачи

Для варианта №20 разрабатывается система газовой котельной. Датчик MQ2 используется как источник аналогового сигнала об условном содержании метана, потенциометр моделирует давление газа в диапазоне 0–5 бар, PIR-датчик фиксирует движение (0 или 1). При s1 > 20 % программа выдаёт команду включения реле, имитирующего газовый клапан, и зажигает светодиод-индикатор; при s1 ≤ 20 % возвращает систему в нормальное состояние.

1. Опрос трёх датчиков с периодом T = 5 с и вывод строки DATA с номером измерения и значениями s1, s2, s3.

2. Формирование THRESHOLD при переходе через порог вверх и NORMAL при возврате; повторная выдача события при неизменном состоянии не требуется.

3. Ручное сохранение строк Serial Monitor в data/log.txt и автоматический запуск обработки после сохранения файла.

4. Хранение измерений и событий в отдельных таблицах readings и events.

5. Выявление значений вне диапазонов: s1 — 0–100 %, s2 — 0–5 бар, s3 — 0 или 1.

6. Выполнение трёх запросов: последние 10 записей показаний; минимум, среднее и максимум; все события по порядку.

Процентная шкала MQ2 является условной нормированной шкалой сигнала, а не измерением объёмной доли метана. Давление получено линейным пересчётом напряжения потенциометра и не является показанием калиброванного манометра. PIR-датчик выдаёт только факт наличия движения.

2. Структура и алгоритм работы системы

2.1. Структурная схема

Система разделена на аппаратную и программную части. Аппаратная часть включает три датчика (MQ2 на входе A1, потенциометр на A2, PIR на D2), Arduino Uno, модуль реле на D5 и белый светодиод с токоограничивающим резистором на D3.

Программная цепочка состоит из Serial Monitor, файла data/log.txt, потока Node-RED и базы SQLite. Wokwi выводит строки в браузере; перенос журнала в локальный файл выполняется вручную. После сохранения файла загрузку выполняет поток. Структурная схема приведена в приложении А.

2.2. Алгоритм функционирования

После включения устанавливаются n = 0 и alarm = false, последовательный порт настраивается на 9600 бод, реле получает HIGH (выключено), светодиод выключается.

В каждом цикле увеличивается n, считываются аналоговые входы A1 и A2 и цифровой вход D2, выводится строка DATA. Если s1 > 20 и alarm = false, признак устанавливается, реле получает LOW, светодиод — HIGH и выводится THRESHOLD. При s1 ≤ 20 и alarm = true реле получает HIGH, светодиод выключается и выводится NORMAL. В конце цикла выдерживается пауза delay(5000). Первая строка DATA выводится сразу после setup(), до первой паузы. Блок-схема приведена в приложении А.

2.3. Протокол обмена

```
DATA n=1 s1=85.73 s2=0.00 s3=0
EVENT n=1 type=THRESHOLD
```

Значения разделяются пробелами; дробная часть записывается через точку с двумя знаками, s3 — целое 0 или 1. n является номером цикла. Время в базе рассчитывается как t_s = n × 5 и является расчётным временем, а не меткой часов реального времени. FAULT создаётся в Node-RED и не передаётся прошивкой.

3. Разработка аппаратной части

3.1. Компоненты и подключение

Таблица 1 — Назначение компонентов и сигналов

Компонент / сигнал	**Подключение**	**Назначение**
Arduino Uno (U1)	5 V, GND	Опрос и управление
MQ2 (U3): AO	A1	Условная шкала метана 0–100 %
Потенциометр PR1: SIG	A2	Давление газа 0–5 бар
PIR (U4): OUT	D2	Движение 0 или 1
Модуль реле (U2): IN	D5	Команда включения (LOW — включено)
Светодиод LED1 и R1	D3 → R1 → анод; катод → GND	Индикация тревоги

Питание модулей соединено с 5 V, общие провода — с GND. Светодиод управляется отдельным цифровым выводом D3 через резистор R1; реле управляется выводом D5 и в модели имитирует газовый клапан. Реальная нагрузка контактов реле в симуляции не подключена.

3.2. Схема EasyEDA и модель Wokwi

На основе проекта EasyEDA подготовлена схема уровня готовых модулей (приложение Б, лист 1) и перечень элементов (приложение Б, лист 2). Листы А4 оформлены в рамках с полями 20 мм слева и 5 мм с остальных сторон; основные надписи — формы 1 и 2 по ГОСТ Р 2.104–2023 [10]. Принятые обозначения: U1 — Arduino Uno, U2 — модуль реле KY-019, U3 — датчик MQ2, U4 — PIR-датчик, PR1 — потенциометр, LED1 — светодиод, R1 — резистор; они едины для схемы, перечня, текста и скетча [11, 12]. Исходный файл проекта EasyEDA (SCH_kyrsovaya_2026-10-29.json) хранится в репозитории в каталоге docs/easyeda/. Шифр документов — КП.5415.20 Э3 и КП.5415.20 ПЭ3. Номинал резистора R1 = 120 Ом на схеме, в перечне, в модели Wokwi и в расчёте один и тот же.

Таблица 2 — Перечень элементов

Обозначение	Наименование	Кол.	Примечание
LED1	White	1	LED0603-RD_WHITE
PR1	Potentiometer	1	WH160-1-104
R1	120 Ом	1	R0603
U1	ARDUINO UNO COPY	1	ARDUINO_UNO_SHIELD
U2	KY-019 5V t	1	KY-019 5V (SRD-05VDC-SL-C)
U3	GAS-SENSOR-MQ2	1	GAS-SENSOR-MQ2
U4	PIR Motion Sensor	1	PIRMOTIONSENSOR

Модель Wokwi (приложение В) собрана из Arduino Uno, макетной платы, датчика газа, потенциометра, PIR-датчика, модуля реле, белого светодиода и резистора 120 Ом. Описание соединений хранится в файле sim/diagram.json, скетч — в firmware/sketch.ino. Проект доступен по ссылке https://wokwi.com/projects/477934511917259777.

3.3. Расчёт резистора и шага АЦП

Расчёт токоограничивающего резистора

Для варианта задан белый светодиод и ток 15 мА. По учебным указаниям принимаются напряжение питания U = 5 В и прямое напряжение светодиода Uf = 3,2 В. Требуемое сопротивление определяется по закону Ома:

$$R = (U - Uf) / I = (5 - 3{,}2) / 0{,}015 = 120 \text{ Ом}.$$

Номинал 120 Ом входит в предложенный ряд, поэтому дополнительное округление не требуется. При выбранном сопротивлении расчётный ток равен 15 мА. Мощность рассеяния:

$$P = I^2 \times R = 0{,}015^2 \times 120 = 0{,}027 \text{ Вт}.$$

Для учебной цепи достаточно резистора с номинальной мощностью не менее 0,125 Вт. Ток светодиода проходит через цифровой вывод D3 Arduino; значение 15 мА не превышает рекомендуемую нагрузку вывода (20 мА). Реле питается от линии 5 V и управляется выводом D5 через собственный модуль.

Расчёт шага АЦП

В учебной модели используется 10-битное преобразование напряжения 0–5 В: 1024 уровня, коды от 0 до 1023. Шаг квантования по напряжению:

$$\Delta U = 5 / 1024 \approx 0{,}00488 \text{ В} = 4{,}88 \text{ мВ}.$$

Первый датчик переводится в условную шкалу 0–100 %. В скетче используется калибровка по коду «чистого воздуха» S1_CLEAN = 210 (раздел 4), поэтому шаг шкалы определяется формулой:

$$\Delta S = (100 - 0) / (1023 - 210) \approx 0{,}123 \%.$$

Один код соответствует примерно 0,12 процентного пункта шкалы. Это разрешение пересчёта, а не погрешность газового датчика. Для условия s1 > 20 % минимальный целый код составляет 373: при коде 372 получается около 19,93 %, при 373 — около 20,05 %. Без калибровки ($\Delta S = 100 / 1023 \approx 0{,}098$ %) порог достигался бы при коде 205.

4. Разработка прошивки

Прошивка написана на C++ с использованием стандартных функций Arduino. Полный текст сохранён в firmware/sketch.ino и приведён в приложении Г. Программа состоит из setup() и loop(); параметры (период, порог, выводы, состояния реле) вынесены в константы в начале файла.

Таблица 3 — Основные операции программы

Операция	**Реализация**
Настройка связи	Serial.begin(9600)
Период опроса	delay(T_MS) в конце loop(), T_MS = 5000
Метан, s1	constrain(100 × (analogRead(A1) − 210) / (1023 − 210), 0, 100)
Давление, s2	analogRead(A2) × 5 / 1023
Движение, s3	digitalRead(2)
Управление реле	D5: LOW — включено; HIGH — выключено
Светодиод	D3: HIGH при тревоге, LOW в норме
Защита от повторных событий	Логический признак alarm

Исходный скетч вычислял s1 по формуле 100 × ADC / 1023. При проверке в Wokwi выяснилось, что датчик MQ2 даже в «чистом воздухе» (ползунок газа в положении 0,1 ppm) выдаёт код АЦП 210, то есть s1 = 20,53 %. Это выше порога 20 %, поэтому возврат в нормальное состояние и событие NORMAL получить было невозможно. Для исправления введена константа S1_CLEAN = 210 — код, измеренный в Wokwi при минимальной концентрации, — и шкала s1 пересчитывается так, чтобы чистому воздуху соответствовало 0 %, а коду 1023 — 100 %; результат ограничивается функцией constrain(). Остальная логика скетча не менялась.

Ползунок газа в Wokwi при каждом запуске симуляции возвращается в положение 400 ppm (код 907, s1 = 85,73 %), поэтому журнал всегда начинается с события THRESHOLD при n = 1. Первая строка DATA выводится сразу после setup(), поэтому соответствие n = 1 и t_s = 5 является учебным приближением: реальный интервал между измерениями немного превышает 5 с из-за времени обработки и передачи строк.

5. Проектирование базы данных

5.1. Модель и структура таблиц

Выбрана SQLite — встраиваемая СУБД, хранящая данные в файле db/variant.db. Дополнительный сервер не требуется; создание таблиц и загрузка выполняются утилитой sqlite3. Структура содержит две таблицы. Показания записываются в длинном формате: одна строка соответствует одному датчику в одном цикле.

Таблица 4 — Поля базы данных

Таблица	**Поля**	**Особенности**
readings	id, n, t_s, sensor, value	id — INTEGER PRIMARY KEY AUTOINCREMENT; n > 0; t_s ≥ 0; sensor только s1, s2 или s3; value — REAL; пара (n, sensor) уникальна
events	id, n, t_s, type, message	id — INTEGER PRIMARY KEY AUTOINCREMENT; type только THRESHOLD, NORMAL или FAULT; message — пояснение, по умолчанию пустая строка

n — номер измерения; t_s — расчётное время в секундах; sensor — обозначение s1, s2 или s3; value — числовое показание. В events поле type содержит THRESHOLD, NORMAL или FAULT, а message — пояснение неисправности.

Ограничения CHECK и UNIQUE защищают от неверных обозначений датчиков и повторной записи канала в одном цикле. Внешнего ключа между таблицами нет: значения n и t_s позволяют сопоставлять показания и события логически, но база не обеспечивает ссылочную целостность. ER-диаграмма дана в приложении Е.

5.2. Контрольные запросы

Первый запрос сортирует показания по n DESC, id DESC и ограничивает результат десятью строками. Второй запрос вычисляет MIN(value), AVG(value), MAX(value) по sensor; среднее округляется до двух знаков. Третий выводит события по n и id. Тексты schema.sql и queries.sql приведены в приложении Д.

6. Загрузка и контроль данных в Node-RED

6.1. Состав потока

Вкладка потока называется «Variant 20 — Gas boiler». В репозитории экспорт находится в nodered/flow.json. Загрузка запускается при изменении файла журнала либо вручную кнопкой inject. Общая структура показана на рисунках 1 и 2.

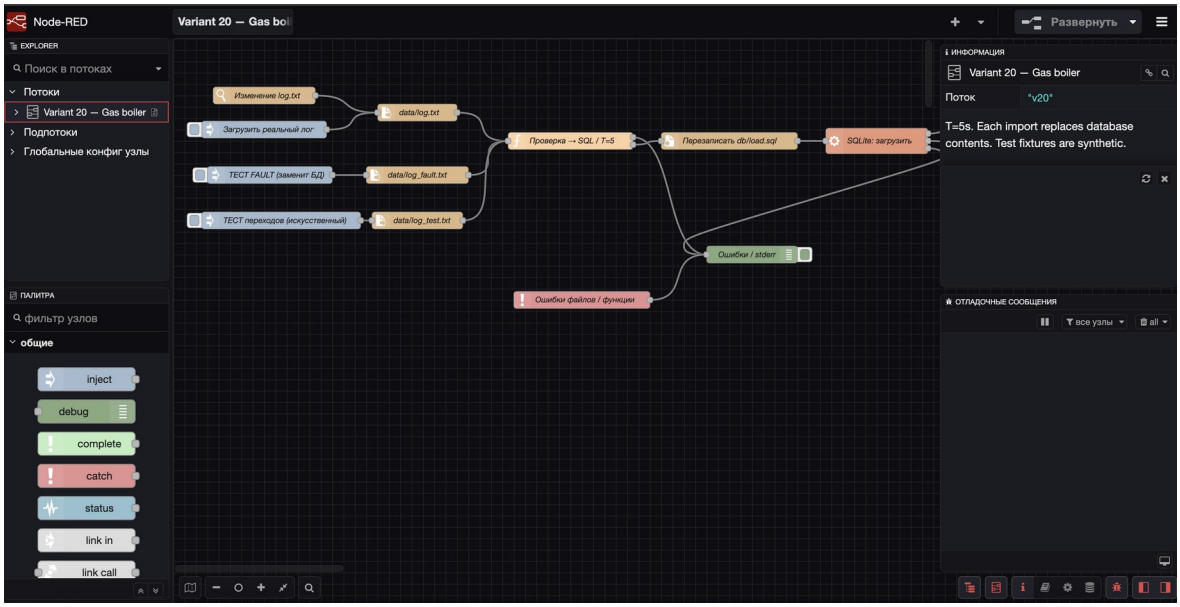


Рисунок 1 — Рабочий поток Node-RED (редактор)

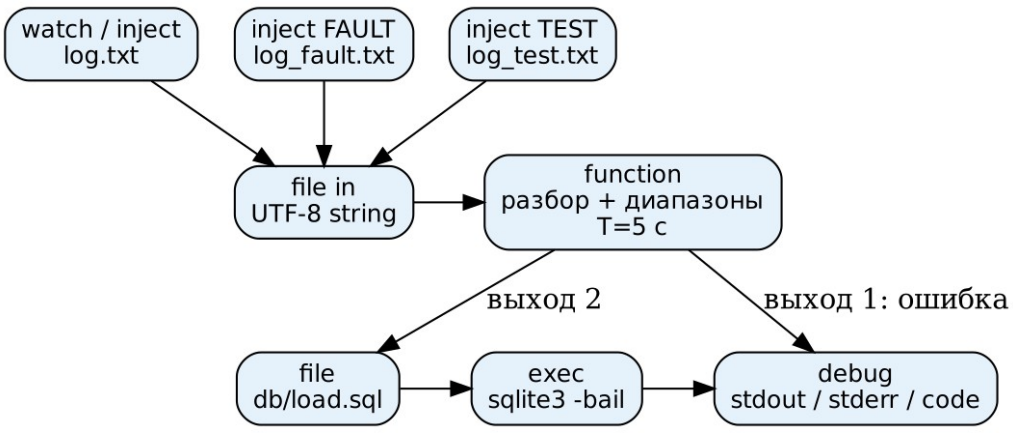


Рисунок 2 — Схема обработки данных в потоке

Таблица 5 — Узлы потока и их назначение

Узел	**Назначение**
Изменение log.txt / watch	Отслеживает сохранение data/log.txt
Загрузить реальный лог / inject	Ручной запуск загрузки основного журнала
data/log.txt / file in	Чтение всего файла в UTF-8
ТЕСТ FAULT, ТЕСТ переходов / inject	Запуск проверок на файлах log_fault.txt и log_test.txt
Проверка → SQL / function	Разбор строк, проверка диапазонов, формирование SQL-скрипта
Перезаписать db/load.sql / file	Запись сформированного SQL в файл
SQLite: загрузить / exec	Запуск sqlite3: чтение schema.sql и load.sql
Ошибки / stderr / debug; catch	Показ ответа, ошибок утилиты и ошибок файлов и функции

6.2. Разбор журнала и запуск

Функция разбирает msg.payload на строки и пары «ключ=значение», удаляет BOM. Допускаются положительные целые n; t_s = n × 5. Для DATA создаются INSERT показаний всех трёх датчиков, повторная строка DATA с тем же n отклоняется; для EVENT принимаются только THRESHOLD и NORMAL, и событие должно следовать за соответствующей строкой DATA. Код функции приведён в приложении Ж.

Числовые значения сравниваются с диапазонами s1: 0–100, s2: 0–5, s3: 0 или 1. При выходе за диапазон значение сохраняется в readings и создаётся FAULT с текстом «датчик sN вне диапазона». Если строка имеет неверный формат (нечисловое или отсутствующее значение, неизвестный тип события, повтор n), весь импорт отклоняется с сообщением об ошибке, а база не изменяется.

Если корректных записей нет, выдаётся ошибка. Иначе второй выход формирует транзакцию BEGIN…COMMIT: очистку таблиц readings и events, сброс счётчиков и вставку записей. Загрузка заменяет снимок журнала, а не накапливает историю, поэтому повторная загрузка того же файла не создаёт дублей. Команда sqlite3 запускается с ключом -bail, чтобы остановиться на первой ошибке.

6.3. Подготовка и запуск

Для работы нужны Node.js, Node-RED и sqlite3. Поток запускается из папки проекта, так как пути data/log.txt и db/variant.db в нём относительные: на macOS и Linux — файлом start-nodered.sh, на Windows — start-nodered.bat. Файл потока передаётся Node-RED аргументом запуска, поэтому повторно импортировать его не нужно.

В браузере открывается http://127.0.0.1:1880. После запуска нажимается «Загрузить реальный лог» либо сохраняется data/log.txt — тогда поток срабатывает автоматически. Просмотр результата выполняется скриптом show-data.sh (show-data.bat) или командой sqlite3 -header -column db/variant.db ".read db/queries.sql". Окно процесса Node-RED должно оставаться открытым.

7. Тестирование и результаты испытаний

7.1. Результаты штатного журнала

Журнал снят в симуляторе Wokwi: строки из Serial Monitor сохранены в data/log.txt без правок. В ходе прогона менялись входные воздействия. При запуске ползунок газа находился в положении 400 ppm, поэтому в цикле n = 1 сформировано THRESHOLD (s1 = 85,73 %). В цикле n = 19 нажата кнопка «Simulate motion» PIR-датчика, s3 = 1. В цикле n = 23 газ уменьшен до минимума (0,1 ppm), s1 = 0 %, выдано NORMAL. В цикле n = 31 ползунок газа выведен на 20 ppm, s1 = 61,62 %, выдано THRESHOLD. В цикле n = 34 повёрнут потенциометр, s2 = 3,02 бар. Журнал содержит 39 циклов, соответствующих расчётному интервалу от 5 до 195 с. После загрузки в базе находятся 117 показаний (39 × 3) и 3 события; FAULT в штатном журнале отсутствует.

Таблица 6 — Результат агрегатного запроса по загруженной базе

Датчик	**Минимум**	**Среднее**	**Максимум**
s1	0.0	62.58	85.73
s2	0.0	0.46	3.02
s3	0.0	0.03	1.0

Вывод запроса 2 (минимум, среднее, максимум)

```
-- 2. Минимум, среднее и максимум каждого датчика
sensor  minimum  average  maximum
------  -------  -------  -------
s1      0.0      62.58    85.73
s2      0.0      0.46     3.02
s3      0.0      0.03     1.0
```

Среднее s1 складывается из трёх участков: 22 цикла по 85,73 %, 8 циклов по 0 % и 9 циклов по 61,62 %. Среднее s3 показывает долю циклов с движением (1 из 39). Агрегатный запрос показывает разброс показаний, но сам по себе не подтверждает точность и калибровку датчиков.

Таблица 7 — События штатного журнала

n	**t_s, с**	**Тип события**
1	5	THRESHOLD
23	115	NORMAL
31	155	THRESHOLD

В начале журнала s1 = 85,73 %, поэтому при исходном alarm = false создаётся THRESHOLD. На измерении 23 s1 снижается до 0 % и создаётся NORMAL; на измерении 31 s1 возрастает до 61,62 % и вновь создаётся THRESHOLD. В последовательных циклах с сохранением состояния новые события не формируются. Результат трёх контрольных запросов показан на рисунке 3.

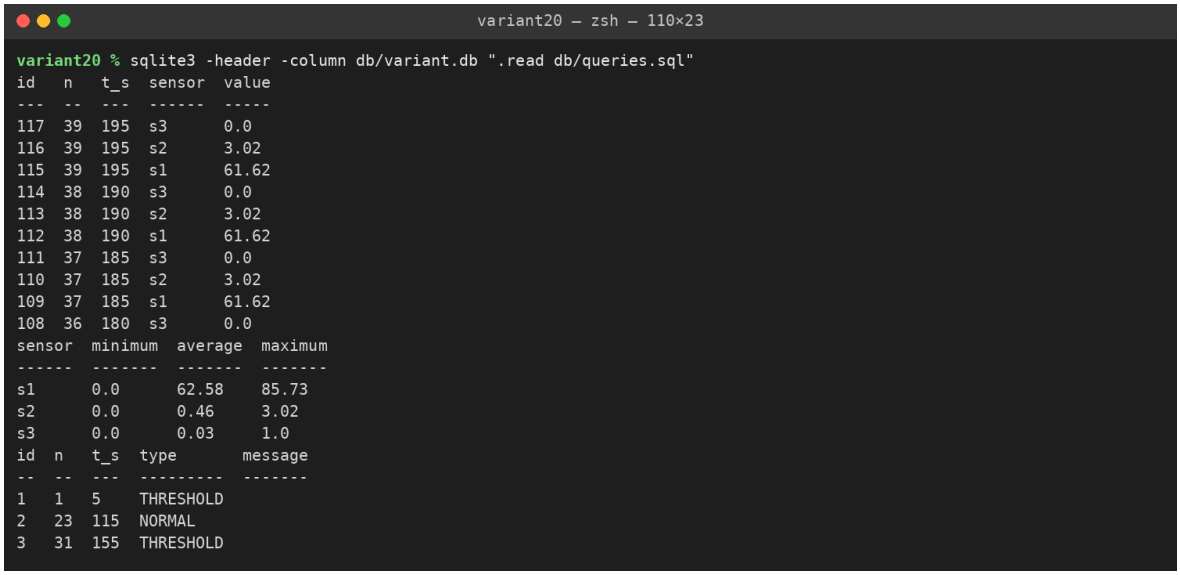


Рисунок 3 — Результат запросов db/queries.sql

7.2. Проверка неисправности и повторной загрузки

Датчики Wokwi не позволяют получить значение s1 больше 100 %, поэтому для проверки обработчика в data/log_fault.txt вручную записана строка:

```
DATA n=1 s1=101.00 s2=2.50 s3=0
```

После загрузки этого файла поток должен создать три показания и событие FAULT для s1. Фактический результат приведён в таблице 8 и на рисунке 4. Строка относится к отдельному искусственному тесту и не является результатом работы Wokwi. После теста нужно снова загрузить основной журнал: на рисунке 4 видно, что события THRESHOLD, NORMAL, THRESHOLD восстанавливаются.

Таблица 8 — Зарегистрированное событие неисправности

id	n	t_s	type	message
1	1	5	FAULT	датчик s1 вне диапазона



Рисунок 4 — Проверка FAULT и возврат к основному журналу

Таблица 9 — Сводка проверок: ожидаемый и фактический результат

Проверка	**Ожидаемый результат**	**Фактический результат**
Съём журнала в Wokwi	Все три события и три датчика в журнале	39 циклов; THRESHOLD при n = 1 и 31, NORMAL при n = 23; s3 = 1 при n = 19; s2 = 3,02 с n = 34
Загрузка data/log.txt	117 показаний и 3 события	117 показаний и 3 события
Повторная загрузка	Количество записей не меняется	Без изменений: 117 и 3
Выход s1 за диапазон (log_fault.txt, s1 = 101)	Одно событие FAULT для s1	Создано событие FAULT «датчик s1 вне диапазона» (id 1, n = 1, t_s = 5)
Искусственный журнал (log_test.txt)	THRESHOLD и NORMAL по порогу	12 показаний; THRESHOLD при n = 2, NORMAL при n = 4
Неверный формат, NaN, повтор n	Импорт отклонён либо создан FAULT	Девять проверок парсера пройдены
Логика скетча на заглушках Arduino	События выдаются однократно	THRESHOLD при n = 2, NORMAL при n = 4
Git	Не менее пяти коммитов и метка v1.0	Пять коммитов и метка v1.0 (приложение И)

Загрузка контрольных журналов выполнена тем же кодом функции, что и в потоке Node-RED (nodered/parser.js), с применением SQLite. Проверка логики скетча на заглушках не заменяет запуск в Wokwi и подтверждает только порядок переходов.

8. Расчёт объёма данных

Расчёт выполняется для периода T = 5 с, трёх датчиков и условного размера одной записи 40 байт. События, индексы, свободные страницы SQLite и служебные накладные расходы в эту оценку не включаются. Используется десятичная единица: 1 МБ = 1 000 000 байт.

Количество циклов за сутки: 86 400 / 5 = 17 280.

N = 17 280 × 3 = 51 840 записей readings.

V = 51 840 × 40 = 2 073 600 байт ≈ 2,07 МБ.

V_{30} = 2 073 600 × 30 = 62 208 000 байт ≈ 62,21 МБ.

Таблица 10 — Расчётный объём показаний

Показатель	**Результат**
Период опроса	5 с
Циклов за сутки	17 280
Записей за сутки	51 840
Объём за сутки	2,0736 МБ ≈ 2,07 МБ
Объём за 30 суток	62,208 МБ
Объём за 365 суток	756,864 МБ ≈ 0,76 ГБ

Расчёт описывает возможное непрерывное накопление показаний. Текущий поток каждый раз перезаписывает базу содержимым одного файла журнала, поэтому реальный размер variant.db не обязан увеличиваться ежедневно на указанную величину. Для долговременного хранения необходимы архивирование журналов или отдельный механизм добавления новых измерений.

При изменении периода T нужно согласованно изменить T_MS в прошивке, константу T в функции Node-RED и расчёт объёма. Уменьшение периода увеличивает число записей; при удвоении периода оно уменьшается в два раза. Выбор периода для реальной котельной требует оценки динамики процесса и времени реакции оборудования.

9. Управление версиями Git

Файлы проекта сохранены в локальном репозитории Git и отправлены в публичный репозиторий GitHub. Основная ветка называется main. Репозиторий: https://github.com/fromhow/-20-. Модель Wokwi: https://wokwi.com/projects/477934511917259777. В репозитории находятся архив проекта (docs/project_archive.zip), исходный код прошивки, схема Wokwi (sim/diagram.json), материалы EasyEDA (схема и перечень элементов в PDF, BOM), поток Node-RED, SQL-скрипты, журналы, инструкция запуска, README и остальные файлы проекта.

Вместо одного общего сохранения применено разделение по этапам работы: структура и схема, модель и прошивка, база и поток, журналы и тесты, записка, презентация и речь. Коммиты созданы при оформлении уже выполненной работы; они фиксируют группы материалов, а не подтверждают разработку в разные дни. Завершённая версия помечена меткой v1.0.

Таблица 11 — Коммиты проекта

№	Хеш	Дата	Сообщение
1	b9d6879	29.10.2026	Добавлены структура проекта, README и принципиальная схема
2	e596545	29.10.2026	Добавлены модель Wokwi и прошивка Arduino
3	5bb25f9	29.10.2026	Добавлены база SQLite и поток Node-RED
4	b4ceb14	29.10.2026	Добавлены журналы Wokwi, тесты и скриншоты проверок
5	—	29.10.2026	Добавлены пояснительная записка, презентация и речь

Хеш последнего коммита не указан, так как он зависит от файла самой записки, находящейся внутри этого коммита.

Таблица 12 — Структура репозитория

Каталог / файл	**Содержимое**
sim/	diagram.json модели Wokwi и ссылка на проект
firmware/	sketch.ino — программа Arduino
db/	schema.sql, queries.sql; variant.db и load.sql создаются автоматически
nodered/	flow.json, parser.js, settings.js
data/	log.txt, log_fault.txt, log_test.txt
scripts/	Проверки парсера и логики скетча
docs/	Схемы, исходник EasyEDA (docs/easyeda), перечень элементов, диаграммы, скриншоты, записка, презентация, речь, архив проекта
start-nodered.sh / .bat	Запуск Node-RED из каталога проекта
show-data.sh / .bat	Вывод трёх контрольных запросов
README.md	Описание варианта, ссылка и порядок запуска

10. Предложения по модернизации

Выполненная система демонстрирует полную цепочку обработки данных, но допускает развитие в нескольких направлениях. Предложения упорядочены по важности для надёжности системы.

1. Калибровка и диагностика датчиков. Выполнить калибровку MQ2 по эталонной смеси вместо условной шкалы, учесть нелинейность датчика и добавить диагностику отсутствующих либо нечисловых показаний.

2. Гистерезис и таймер. Реализовать гистерезис порога (разные значения включения и выключения) и опрос по millis(), чтобы исключить частые переключения реле при шуме и уточнить период измерений.

3. Накопление данных. Добавить идентификатор сеанса и накопительную загрузку с защитой от дублей; заменить ручное копирование Serial Monitor передачей данных с подтверждением доставки.

4. Время и аналитика. Заменить расчётное время $t_s = n \times 5$ меткой реального времени и добавить в Node-RED вывод графиков показаний и событий.

5. Безопасность. Для реального применения использовать сертифицированные датчики и схему аварийного отключения газа, независимую от программы Arduino.

Выводы

Разработана виртуальная система сбора и учёта данных для варианта №20 «Газовая котельная». Модель объединяет Arduino Uno, датчик MQ2, потенциометр, PIR-датчик, реле и светодиод. Подготовлены структурная схема, блок-схема алгоритма, принципиальная схема и перечень элементов.

Прошивка выводит DATA с периодом 5 с и формирует THRESHOLD и NORMAL при смене порогового состояния s1 > 20 %. При проверке в Wokwi выявлено, что без калибровки датчик MQ2 не опускается ниже 20,53 %; введена калибровка по коду «чистого воздуха», после чего получен журнал со всеми тремя событиями. Выполнены расчёты: R = 120 Ом, шаг АЦП 4,88 мВ и шаг шкалы s1 около 0,123 %.

В SQLite созданы таблицы readings и events. Настроен поток Node-RED с автоматическим и ручным запуском загрузки. По журналу из 39 циклов в базе получены 117 показаний и три пороговых события; повторная загрузка дублей не создаёт. Отдельная проверка продемонстрировала регистрацию FAULT для значения s1 вне диапазона.

Расчётная оценка составляет 2,07 МБ показаний в сутки и 62,21 МБ за 30 суток при непрерывном накоплении. Для текущей загрузки снимками эта оценка является ориентиром для модернизации, а не измеренным ростом файла базы. Проект сохранён в Git и опубликован на GitHub.

Список используемых сокращений

Сокращение	**Расшифровка**
АЦП	Аналого-цифровой преобразователь
БД	База данных
ИИС	Интеллектуальная интегрированная система
СУБД	Система управления базами данных
BOM	Bill of Materials — перечень элементов
COM	Общий контакт реле
NO	Нормально разомкнутый контакт реле
NC	Нормально замкнутый контакт реле
MQ2	Датчик горючих газов (метан, дым)
PIR	Пассивный инфракрасный датчик движения
SQL	Язык структурированных запросов
UART	Универсальный асинхронный приёмопередатчик
USB	Универсальная последовательная шина
DATA	Сообщение с измерениями
EVENT	Сообщение о смене состояния
FAULT	Событие недопустимого показания
THRESHOLD	Переход в пороговое состояние
NORMAL	Возврат в нормальное состояние

Список используемой литературы

1. Методические указания к курсовому проекту ПМ.02. «Титульник и задание ПМ 02». Материалы Петровского колледжа, предоставленные к работе. Вариант №20.

2. Arduino. Language Reference. URL: https://docs.arduino.cc/language-reference/. Справочный ресурс по функциям Arduino.

3. Wokwi. Documentation. URL: https://docs.wokwi.com/. Справочный ресурс по виртуальным компонентам.

4. Node-RED. Documentation. URL: https://nodered.org/docs/. Справочный ресурс по узлам и потокам.

5. SQLite. Documentation. URL: https://www.sqlite.org/docs.html. Справочный ресурс по типам, запросам и транзакциям.

6. Чакон С., Штрауб Б. Pro Git. URL: https://git-scm.com/book/ru/v2. Электронная книга.

7. EasyEDA. Documentation. URL: https://docs.easyeda.com/. Справочный ресурс по схемам и экспорту.

8. Репозиторий проекта варианта №20. URL: https://github.com/fromhow/-20-.

9. Проект Wokwi варианта №20. URL: https://wokwi.com/projects/477934511917259777. Рабочая модель проекта.

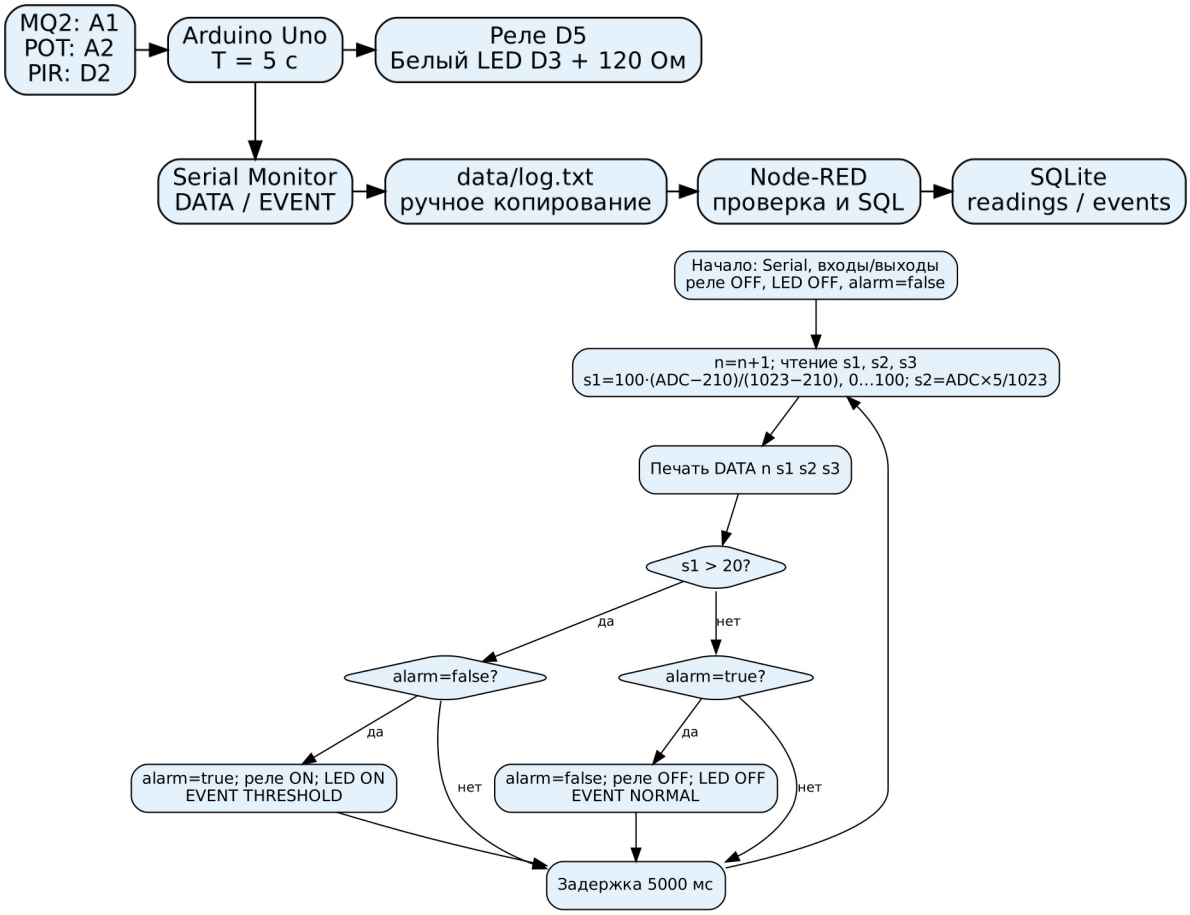
10. ГОСТ Р 2.104–2023. Единая система конструкторской документации. Основные надписи. Введён 01.03.2024. Формы 1 и 2, приложение Б.

11. ГОСТ 2.702–2011. Единая система конструкторской документации. Правила выполнения электрических схем. Раздел 5: принципиальные схемы и перечни элементов.

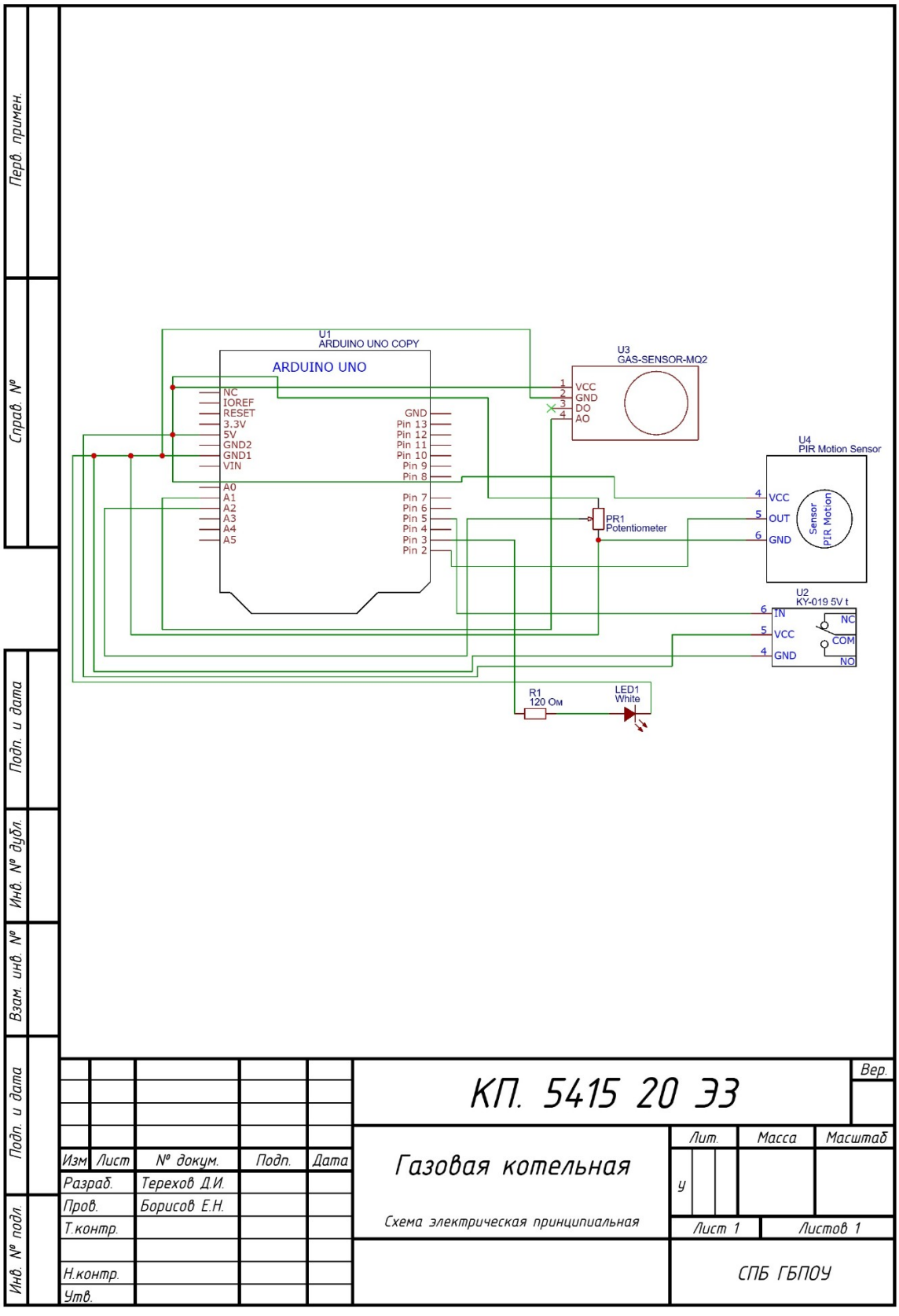
12. ГОСТ 2.710–81. Единая система конструкторской документации. Обозначения буквенно-цифровые в электрических схемах.

13. ГОСТ 2.301–68. Единая система конструкторской документации. Форматы. Формат А4 — 210 × 297 мм.

Приложение А. Структурная схема и алгоритм



Приложение Б. Схема и спецификация



					КП. 5415 20 ЭЗ	Вер.
Изм	Лист	№ докум.	Подп.	Дата	Газовая котельная	Лит. / Масса / Масштаб
Разраб.		Терехов Д.И.				У
Пров.		Борисов Е.Н.			Схема электрическая принципиальная	Лист 1 / Листов 1
Т.контр.						
Н.контр.					СПБ ГБПОУ	
Утв.						

Перв. примен. | Справ. № | Подп. и дата | Инв. № дубл. | Взам. инв. № | Подп. и дата | Инв. № подл.

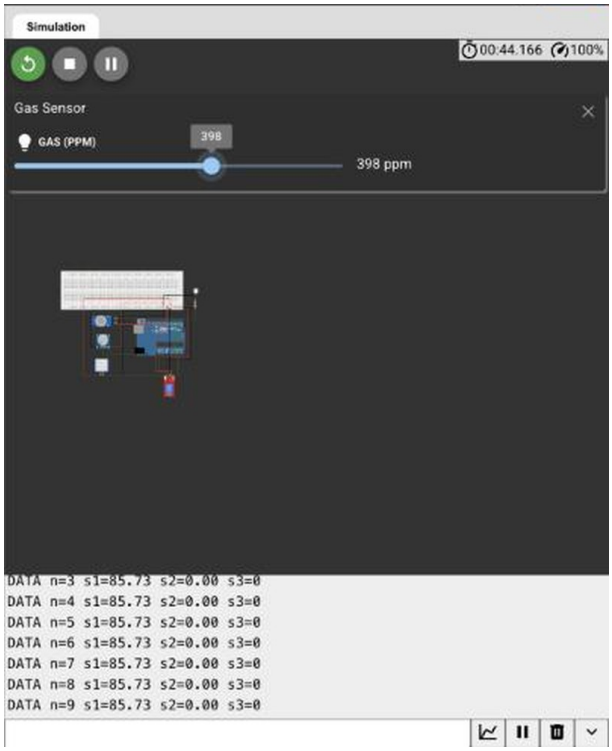
Поз. обозначение	Наименование	Кол.	Примечание
	Светодиоды		
LED1	White	1	LED0603-RD_WHITE
	Потенциометры		
PR1	Potentiometer	1	WH160-1-104
	Резисторы		
R1	120 Ом	1	R0603
	Микросхемы, неразборные узлы		
U1	ARDUINO UNO COPY	1	ARDUINO_UNO_SHIEL
U2	KY-019 5V t	1	KY-019 5V
U3	GAS-SENSOR-MQ2	1	GAS-SENSOR-MQ2
U4	PIR Motion Sensor	1	PIRMOTIONSENSOR

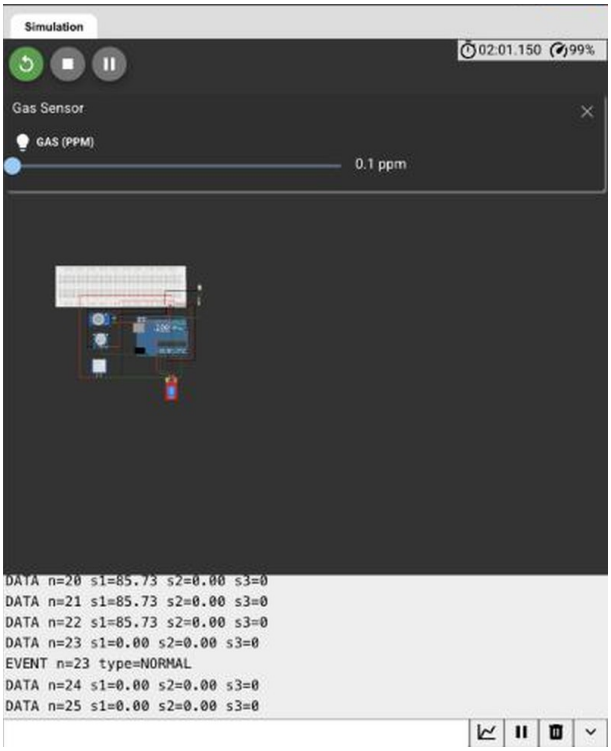
Перв. примен.
Справ. №
Подп. и дата
Инв. № дубл.
Взам. инв. №
Подп. и дата
Инв. № подл.

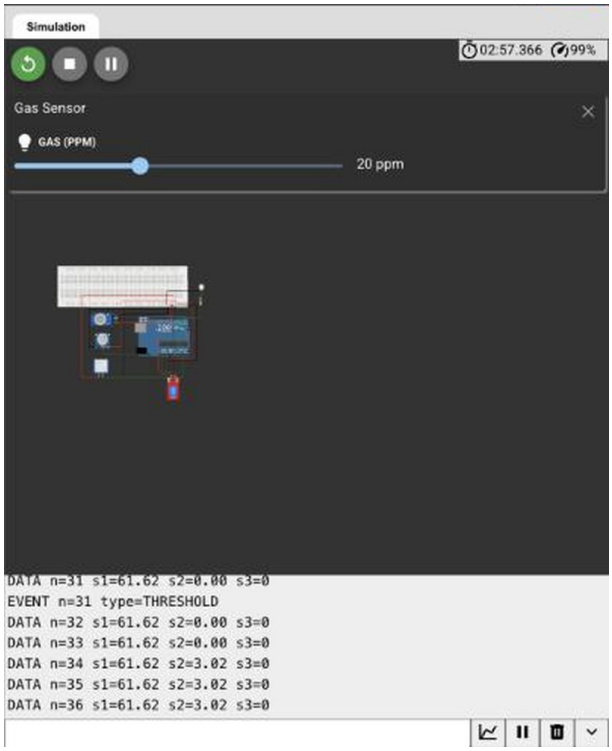
Изм	Лист	№ докум.	Подп.	Дата	КП. 5415 20 ПЭ3	Вер.
Разраб.		Терехов Д.И.			Газовая котельная	Лит. / Лист / Листов
Пров.		Борисов Е.Н.				у / 1 / 1
Нач. сект.						СПБ ГБПОУ
Н.контр.					Перечень элементов	
Утв.						

Приложение В. Схема в Wokwi

Модель Wokwi в трёх состояниях: слева вверху — тревога при газе 398 ppm (s1 = 85,73 %); справа вверху — норма при газе 0,1 ppm (s1 = 0 %, EVENT n=23 type=NORMAL); внизу — повторная тревога (EVENT n=31 type=THRESHOLD), s2 = 3,02 бар. Показаны окно симуляции и Serial Monitor.







Приложение Г. Листинг скетча

```
const unsigned long T_MS = 5000;

const int PIN_S1 = A1;
const int PIN_S2 = A2;
const int PIN_S3 = 2;
const int PIN_RELAY = 5;
const int PIN_LED = 3;

const float THRESHOLD = 20.0;

// Калибровка MQ2: в Wokwi в "чистом воздухе" (0.1 ppm) код АЦП = 210
const int S1_CLEAN = 210;

const int RELAY_ON = LOW;
const int RELAY_OFF = HIGH;

unsigned long n = 0;
bool alarm = false;

void setup() {
  Serial.begin(9600);

  pinMode(PIN_S1, INPUT);
  pinMode(PIN_S2, INPUT);
  pinMode(PIN_S3, INPUT);

  digitalWrite(PIN_RELAY, RELAY_OFF);
  pinMode(PIN_RELAY, OUTPUT);

  digitalWrite(PIN_LED, LOW);
  pinMode(PIN_LED, OUTPUT);
}

void loop() {
  n++;

  float s1 = constrain(100.0 * (analogRead(PIN_S1) - S1_CLEAN) / (1023.0 - S1_CLEAN), 0.0, 100.0);

  float s2 = 5.0 * analogRead(PIN_S2) / 1023.0;

  int s3 = digitalRead(PIN_S3);

  Serial.print("DATA n=");
  Serial.print(n);
  Serial.print(" s1=");
  Serial.print(s1, 2);
  Serial.print(" s2=");
  Serial.print(s2, 2);
  Serial.print(" s3=");
  Serial.println(s3);

  bool condition = s1 > THRESHOLD;

  if (condition && !alarm) {
    alarm = true;

    digitalWrite(PIN_RELAY, RELAY_ON);
    digitalWrite(PIN_LED, HIGH);

    Serial.print("EVENT n=");
    Serial.print(n);
    Serial.println(" type=THRESHOLD");
  } else if (!condition && alarm) {
    alarm = false;

    digitalWrite(PIN_RELAY, RELAY_OFF);
    digitalWrite(PIN_LED, LOW);

    Serial.print("EVENT n=");
    Serial.print(n);
    Serial.println(" type=NORMAL");
  }

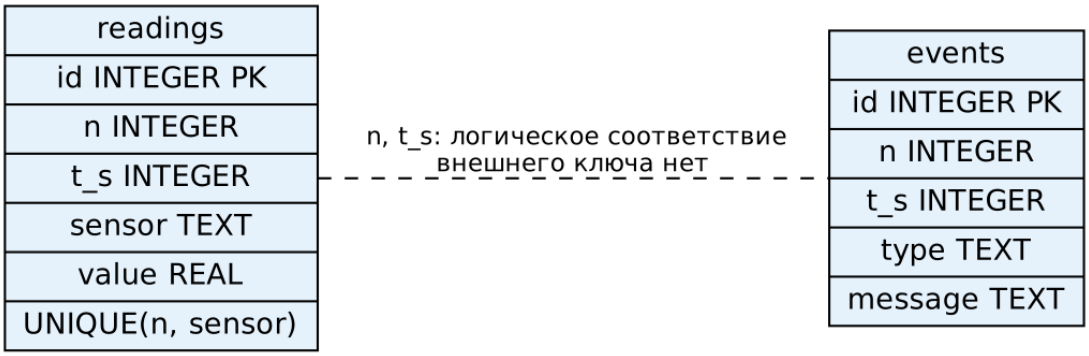
  delay(T_MS);
}
```

Приложение Д. Схема и запросы SQL

```
-- db/schema.sql
-- Вариант 20. Схема базы данных SQLite: показания (readings) и события
(events).
PRAGMA foreign_keys = ON;
CREATE TABLE IF NOT EXISTS readings (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 n INTEGER NOT NULL CHECK(n>0),
 t_s INTEGER NOT NULL CHECK(t_s>=0),
 sensor TEXT NOT NULL CHECK(sensor IN ('s1','s2','s3')),
 value REAL NOT NULL,
 UNIQUE(n,sensor)
);
CREATE TABLE IF NOT EXISTS events (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 n INTEGER NOT NULL CHECK(n>0),
 t_s INTEGER NOT NULL CHECK(t_s>=0),
 type TEXT NOT NULL CHECK(type IN ('THRESHOLD','NORMAL','FAULT')),
 message TEXT NOT NULL DEFAULT ''
);
CREATE INDEX IF NOT EXISTS idx_readings_sensor_time ON readings(sensor,t_s);
CREATE INDEX IF NOT EXISTS idx_events_time ON events(t_s,id);

-- db/queries.sql
-- 1. Последние 10 записей показаний (десять отдельных строк, а не десять
циклов)
SELECT id,n,t_s,sensor,value FROM readings ORDER BY n DESC,id DESC LIMIT 10;
-- 2. Минимум, среднее и максимум каждого датчика
SELECT sensor,MIN(value) AS minimum,ROUND(AVG(value),2) AS average,MAX(value) AS
maximum
FROM readings GROUP BY sensor ORDER BY sensor;
-- 3. Все события по порядку
SELECT id,n,t_s,type,message FROM events ORDER BY n,id;
```

Приложение E. ER-диаграмма



Приложение Ж. Код функции потока Node-RED

```
// nodered/parser.js
// T must match T_MS/1000 in firmware/sketch.ino.
const T = 5;
const RANGES = {s1:[0,100],s2:[0,5],s3:[0,1]};
const quote = s => "'" + String(s).replace(/'/g,"''") + "'";
const text = String(msg.payload).replace(/^\uFEFF/, '');
const sql = [];
let rows = 0;
try {
 const seen = new Set();
 text.split(/\r?\n/).forEach((raw,index) => {
  const line = raw.trim();
  if (!line) return;
  const parts = line.split(/\s+/);
  const kind = parts.shift();
  if (!['DATA','EVENT'].includes(kind)) throw Error(`line ${index+1}: expected DATA or
EVENT`);
  const f = {};
  parts.forEach(p => {
   const m = /^([a-z][a-z0-9]*)=(\S+)$/i.exec(p);
   if (!m || Object.prototype.hasOwnProperty.call(f,m[1])) throw Error(`line ${index+1}: bad
or repeated field`);
   f[m[1]]=m[2];
  });
  if (!/^\d+$/.test(f.n || '')) throw Error(`line ${index+1}: invalid n`);
  const n=Number(f.n), t=n*T;
  if (!Number.isSafeInteger(n) || n<1 || !Number.isSafeInteger(t)) throw Error(`line $
{index+1}: n out of bounds`);
  if (kind==='DATA') {
   if (seen.has(n)) throw Error(`line ${index+1}: duplicate DATA n=${n}`);
   seen.add(n);
   ['s1','s2','s3'].forEach(s => {
    if (!/^[+-]?(?:\d+(?:\.\d*)?|\.\d+)$/.test(f[s] || '')) throw Error(`line ${index+1}:
invalid ${s}`);
    const v=Number(f[s]);
    if (!Number.isFinite(v)) throw Error(`line ${index+1}: non-finite ${s}`);
    sql.push(`INSERT INTO readings(n,t_s,sensor,value) VALUES(${n},${t},${quote(s)},${v});`);
    rows++;
    const bad=v<RANGES[s][0] || v>RANGES[s][1] || (s==='s3' && ![0,1].includes(v));
    if (bad) sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t},'FAULT',$
{quote(`датчик ${s} вне диапазона`)});`);
   });
  } else {
   if (!seen.has(n)) throw Error(`line ${index+1}: EVENT must follow its DATA`);
   if (!['THRESHOLD','NORMAL'].includes(f.type)) throw Error(`line ${index+1}: invalid event
type`);
   sql.push(`INSERT INTO events(n,t_s,type,message) VALUES(${n},${t},${quote(f.type)},'');`);
  }
 });
 if (!rows) throw Error('no DATA records');
} catch(e) {
 node.error(e.message,msg);
 return [{payload:'IMPORT REJECTED: '+e.message},null];
}
const head="BEGIN;\nDELETE FROM readings;\nDELETE FROM events;\nDELETE FROM sqlite_sequence
WHERE name IN ('readings','events');\n";
return [null,{payload:head+sql.join('\n')+'\nCOMMIT;\n'}];
```

Приложение 3. Журналы и результаты запросов

Штатный журнал data/log.txt (39 циклов, строки Serial Monitor без правок)

```
DATA n=1 s1=85.73 s2=0.00 s3=0
EVENT n=1 type=THRESHOLD
DATA n=2 s1=85.73 s2=0.00 s3=0
DATA n=3 s1=85.73 s2=0.00 s3=0
DATA n=4 s1=85.73 s2=0.00 s3=0
DATA n=5 s1=85.73 s2=0.00 s3=0
DATA n=6 s1=85.73 s2=0.00 s3=0
DATA n=7 s1=85.73 s2=0.00 s3=0
DATA n=8 s1=85.73 s2=0.00 s3=0
DATA n=9 s1=85.73 s2=0.00 s3=0
DATA n=10 s1=85.73 s2=0.00 s3=0
DATA n=11 s1=85.73 s2=0.00 s3=0
DATA n=12 s1=85.73 s2=0.00 s3=0
DATA n=13 s1=85.73 s2=0.00 s3=0
DATA n=14 s1=85.73 s2=0.00 s3=0
DATA n=15 s1=85.73 s2=0.00 s3=0
DATA n=16 s1=85.73 s2=0.00 s3=0
DATA n=17 s1=85.73 s2=0.00 s3=0
DATA n=18 s1=85.73 s2=0.00 s3=0
DATA n=19 s1=85.73 s2=0.00 s3=1
DATA n=20 s1=85.73 s2=0.00 s3=0
DATA n=21 s1=85.73 s2=0.00 s3=0
DATA n=22 s1=85.73 s2=0.00 s3=0
DATA n=23 s1=0.00 s2=0.00 s3=0
EVENT n=23 type=NORMAL
DATA n=24 s1=0.00 s2=0.00 s3=0
DATA n=25 s1=0.00 s2=0.00 s3=0
DATA n=26 s1=0.00 s2=0.00 s3=0
DATA n=27 s1=0.00 s2=0.00 s3=0
DATA n=28 s1=0.00 s2=0.00 s3=0
DATA n=29 s1=0.00 s2=0.00 s3=0
DATA n=30 s1=0.00 s2=0.00 s3=0
DATA n=31 s1=61.62 s2=0.00 s3=0
EVENT n=31 type=THRESHOLD
DATA n=32 s1=61.62 s2=0.00 s3=0
DATA n=33 s1=61.62 s2=0.00 s3=0
DATA n=34 s1=61.62 s2=3.02 s3=0
DATA n=35 s1=61.62 s2=3.02 s3=0
DATA n=36 s1=61.62 s2=3.02 s3=0
DATA n=37 s1=61.62 s2=3.02 s3=0
DATA n=38 s1=61.62 s2=3.02 s3=0
DATA n=39 s1=61.62 s2=3.02 s3=0
```

Журнал проверки неисправности data/log_fault.txt

```
DATA n=1 s1=101.00 s2=2.50 s3=0
```

Результат загрузки: в readings три показания (n = 1), в events одно событие: 1, 1, 5, FAULT, датчик s1 вне диапазона.

Результат запросов db/queries.sql после загрузки data/log.txt

```
-- 1. Последние 10 записей показаний
id   n   t_s  sensor  value
---  --  ---  ------  -----
117  39  195  s3      0.0
116  39  195  s2      3.02
115  39  195  s1      61.62
114  38  190  s3      0.0
113  38  190  s2      3.02
112  38  190  s1      61.62
111  37  185  s3      0.0
```

```
110  37  185  s2      3.02
109  37  185  s1      61.62
108  36  180  s3      0.0

-- 2. Минимум, среднее и максимум каждого датчика
sensor  minimum  average  maximum
------  -------  -------  -------
s1      0.0      62.58    85.73
s2      0.0      0.46     3.02
s3      0.0      0.03     1.0

-- 3. Все события по порядку
id  n   t_s  type       message
--  --  ---  ---------  -------
1   1   5    THRESHOLD
2   23  115  NORMAL
3   31  155  THRESHOLD
```

Приложение И. История Git и структура репозитория

Публичный репозиторий проекта: https://github.com/fromhow/-20-.

История коммитов (git log) и метка

```
$ git log --reverse --format="%h  %ad  %s" --date=short
b9d6879  2026-10-29  Добавлены структура проекта, README и принципиальная схема
e596545  2026-10-29  Добавлены модель Wokwi и прошивка Arduino
5bb25f9  2026-10-29  Добавлены база SQLite и поток Node-RED
b4ceb14  2026-10-29  Добавлены журналы Wokwi, тесты и скриншоты проверок
(хеш)   2026-10-29  Добавлены пояснительная записка, презентация и речь

$ git tag
v1.0

$ git remote -v
origin  https://github.com/fromhow/-20-.git (push)
```

Хеш последнего коммита определяется после его создания, так как коммит содержит саму записку.

Структура репозитория

```
firmware/sketch.ino      прошивка Arduino
sim/                     diagram.json и ссылка на проект Wokwi
db/                      schema.sql, queries.sql
nodered/                 flow.json, parser.js, settings.js
data/                    log.txt, log_fault.txt, log_test.txt
scripts/                 проверки парсера и логики скетча
docs/                    схема EasyEDA, исходник EasyEDA (docs/easyeda), BOM, диаграммы,
                         скриншоты, записка, презентация, речь,
                         project_archive.zip — архив проекта
README.md                описание и порядок запуска
start-nodered.sh / .bat  запуск Node-RED из папки проекта
show-data.sh / .bat      вывод контрольных запросов
```